Artificial Intelligence

The 80/20 Revision Guide

Part IB — Cambridge CST — Paper 7 (Dr S. Holden)

How this guide is organised

Two questions a year. Across **2018–2025 (16 questions)** the per-question tag frequencies are:

Topic	Appearances	Share
Constraint satisfaction (CSP)	5	31%
Neural networks / backprop / gradient descent / activations	6 (cluster)	∼38%
Heuristic search (A*)	4	25%
Planning: state-variable representation	4	25%
Boolean satisfiability (SAT)	3	19%
Planning graphs / partial-order planning	3	19%
Uninformed search; game playing; knowledge representation	rare	<6%

80/20 verdict. Four themes own this paper: **CSP (+SAT)**, the **neural-network cluster**, **heuristic search (A*)**, and **planning** (especially the state-variable representation). Strikingly, **uninformed search, minimax/α-β game playing, and knowledge representation almost never appear** in recent papers — know them lightly, don't over-invest.

This guide focuses on neural networks and heuristic search (the high-yield topics not yet in your notes). For **planning (STRIPS, partial-order, state-variable representation) and CSP encoding**, your companion notes `planning_strips_csp.pdf` already cover the constructions in depth — this guide adds the *CSP algorithms* and *planning graphs* and points back to those notes for the rest.

---

1 Neural networks (Tier 1 — biggest cluster, most new value)

Exam-critical

[title=From perceptron to MLP] **Supervised learning** fits a function to labelled data (*curve fitting* for regression, *classification* for discrete labels). A **perceptron** computes $y = g(\mathbf{w}\cdot\mathbf{x}+b)$ — a *linear* decision boundary. The perceptron learning rule nudges weights on misclassification. **Limitation:** a single layer can only separate *linearly separable* data (it cannot learn XOR) — this motivates the **multilayer perceptron (MLP)**: hidden layers of non-linear units compose to represent non-linear boundaries.

Exam-critical

[title=Activation & error functions] **Activation** g must be non-linear (else layers collapse to one linear map):

$$\text{sigmoid } \sigma(x) = \tfrac{1}{1+e^{-x}} \quad (\sigma' = \sigma(1-\sigma)), \qquad \tanh, \qquad \text{ReLU } \max(0,x).$$

Error (loss) for outputs y_k vs targets t_k: sum-squared error $E = \frac{1}{2}\sum_k (y_k - t_k)^2$ (regression) or cross-entropy (classification). Training = minimise E over the weights.

Exam-critical

[title=Gradient descent & backpropagation (the derivation examiners want)] **Gradient descent:** update each weight against the gradient, $w \leftarrow w - \eta \dfrac{\partial E}{\partial w}$ (η the learning rate). **Backpropagation** computes those gradients efficiently by the chain rule:

1. **Forward pass:** compute each unit's weighted input $\text{in}_j = \sum_i w_{ij} a_i$ and activation $a_j = g(\text{in}_j)$.
2. **Output layer:** $\delta_k = (a_k - t_k)\, g'(\text{in}_k)$.
3. **Backpropagate:** for a hidden unit j, $\delta_j = g'(\text{in}_j) \sum_k w_{jk}\, \delta_k$ (sum over units it feeds).
4. **Update:** $\Delta w_{ij} = -\eta\, a_i\, \delta_j$.

The whole point: $\partial E / \partial w_{ij} = a_i \delta_j$, and the δ's are reused layer-by-layer, so one backward pass gives *all* gradients.

Exam trap

Be ready to **derive** δ for one weight from E using the chain rule, and to state *why* a non-linear activation is essential and *why* a single perceptron fails on XOR. Know $\sigma' = \sigma(1-\sigma)$ cold — it appears in every backprop derivation. Watch sign conventions ($a_k - t_k$ vs $t_k - a_k$) and keep them consistent.

2 Heuristic search & A* (Tier 1 — 25%)

Exam-critical

A* expands the node with least $f(n) = g(n) + h(n)$: g = cost so far, h = heuristic estimate to goal.

- **Admissible:** $h(n) \leq h^*(n)$ (never overestimates) $\Rightarrow$ **tree-search** A* is *optimal*.
- **Consistent (monotone):** $h(n) \leq c(n, a, n') + h(n') \Rightarrow$ **graph-search** A* is optimal (f non-decreasing along any path). Consistency $\Rightarrow$ admissibility.

A* is optimally efficient (no algorithm expands fewer nodes for a given h). Be ready to **prove optimality**: if a suboptimal goal G_2 were expanded before optimal G, admissibility gives $f(G_2) > f(G) \geq f(n)$ for a node n on the optimal path — contradiction.

Worth knowing

Variants: IDA* (iterative-deepening on the f-cost bound — linear memory), **recursive best-first search** (RBFS, linear memory, backs up f-values). **Local search** when only the goal state matters: *hill-climbing* (greedy, gets stuck in local optima — escape via random restarts / simulated annealing), and **gradient descent** as continuous-space hill-climbing (links straight to NN training). **Designing heuristics:** relax the problem (drop constraints) for an admissible h — see `planning_strips_csp.pdf`'s heuristics notes.

3 Constraint satisfaction & SAT (Tier 1 — 31% + 19%)

Exam-critical

[title=CSP solving algorithms] A CSP is ⟨variables, domains, constraints⟩ (formulation/encoding covered in `planning_strips_csp.pdf`). Solve by **backtracking** DFS over assignments, boosted by:

- **Forward checking:** after assigning a variable, prune inconsistent values from the domains of its unassigned neighbours; backtrack when a domain empties.
- **Arc consistency (AC-3):** maintain a queue of arcs; *revise* arc (X_i, X_j) by deleting values of X_i with no support in X_j; if a domain changes, re-enqueue its neighbours' arcs. Runs to a fixpoint; can be run as preprocessing or interleaved (MAC).
- **Heuristics:** *minimum-remaining-values* (MRV) and *degree* for variable order; *least-constraining-value* for value order.
- **Backjumping:** on failure, jump back to the variable that caused the conflict (not just the previous one).

Worth knowing

SAT is the Boolean special case (variables true/false, clauses in CNF). It's solved by **DPLL** (unit propagation, pure-literal elimination, case split) — see `logic_8020.pdf` for the full algorithm. Many AI problems (planning, CSP) are encoded *into* SAT and handed to a solver; the AI exam often pairs CSP with SAT.

4 Planning (Tier 1 — state-variable rep 25%, graphs 19%)

Worth knowing

Your `planning_strips_csp.pdf` already covers, in depth: **STRIPS** (⟨Pre, Add, Del⟩, closed-world, $s' = (s \setminus \text{Del}) \cup \text{Add}$), **partial-order planning** (causal links, threats, promotion/demotion, counting linearisations), the **state-variable representation** (finite-domain variables, rigid vs fluent relations — the most-examined planning topic), and **planning → CSP**. Revise those from that document.

Exam-critical

[title=Planning graphs / GRAPHPLAN (the bit to add)] A **planning graph** alternates *state levels* (literals) and *action levels*, built forward in polynomial time. It records **mutex** (mutually exclusive) relations:

- actions are mutex if one deletes a precondition/effect of the other (interference), or their preconditions are mutex (competing needs);
- literals are mutex if all ways of achieving them are pairwise mutex.

GRAPHPLAN: expand the graph until all goal literals appear non-mutex at some level, then *extract* a plan by backward search; if extraction fails, expand another level. The graph also yields an admissible heuristic (the level at which a goal first appears is a lower bound on cost) for heuristic planners.

5 Lighter topics (Tier 2/3 — know, don't over-invest)

Worth knowing

Uninformed search: BFS (complete, optimal for unit costs, $O(b^d)$ space), DFS ($O(bm)$ space, not optimal), **iterative deepening** (DFS's space with BFS's optimality — the practical default). Tree vs graph search (graph search keeps an explored set to avoid revisiting).

Game playing: minimax for two-player zero-sum games; **α-β pruning** prunes branches that cannot affect the result (best-case halves the effective depth, $O(b^{m/2})$, given good move ordering). Be able to *trace* α-β on a small tree.

Knowledge representation: semantic networks, frames & rules, inheritance; **forward chaining** (data-driven) vs **backward chaining** (goal-driven); first-order logic and **situation calculus** (the Result(a, s) fluent framing that STRIPS replaces).

Exam checklist

Exam-critical

1. **Neural networks:** perceptron = linear boundary (fails XOR) → MLP; non-linear activation essential; $\sigma' = \sigma(1-\sigma)$; SSE loss; **gradient descent** $w \leftarrow w - \eta\, \partial E/\partial w$; **backprop** (forward pass; $\delta_k = (a_k - t_k)g'$; $\delta_j = g' \sum w\delta$; $\Delta w_{ij} = -\eta a_i \delta_j$) — be able to derive it.
2. **A*:** $f = g + h$; *admissible* $h \le h^* \Rightarrow$ tree-search optimal; *consistent* $h(n) \le c + h(n') \Rightarrow$ graph-search optimal; prove optimality; IDA*/RBFS; hill-climbing local optima.
3. **CSP:** backtracking + forward checking + **AC-3** (revise arcs to a fixpoint) + MRV/degree/LCV + backjumping. SAT via DPLL (see logic guide); AI often pairs CSP with SAT.
4. **Planning:** STRIPS / partial-order / **state-variable representation** (from `planning_strips_csp.pdf`); **planning graphs** (mutexes: interference, competing needs) and **GRAPHPLAN** (expand to non-mutex goals, then extract); planning-graph level = admissible heuristic.
5. **Lighter:** BFS/DFS/IDS (completeness/optimality/complexity); minimax + α-β ($O(b^{m/2})$, trace it); forward vs backward chaining; situation calculus.